from the place where you call using until the end of the module (if you're in a module definition) or the end of the file (if you're at the top level)

In the limited scope where it's active, a Refinement is just as good as an Open Class or a Monkeypatch. It can define new methods, redefine existing methods, include or prepend modules, and generally do anything that a regular Open Class can do. Code in an active Refinement takes precedence over code in the refined class, and also over code in modules that are included or prepended by the class. Refining a class is like slapping a patch right onto the original code of the class.

On the other hand, because they're not global, Refinements don't have the issues that you experienced in *The Problem with Open Classes*, on page 15. You can apply a Refinement to a few selected areas of your code, and the rest of your code will stick with the original unrefined class—so there aren't many chances that you'll break your system by inadvertently impacting unrelated code. However, this local quality of Refinements has the potential to surprise you, as you're about to find out.

Refinement Gotchas

Look at this code:

object_model/refinements_gotcha.rb

```
class MyClass
  def my_method
    "original my_method()"
  end

  def another_method
    my_method
  end
end

module MyClassRefinement
  refine MyClass do
    def my_method
      "refined my_method()"
    end
  end
end

using MyClassRefinement
MyClass.new.my_method       # =>  "refined my_method()"
MyClass.new.another_method  # =>  "original my_method()"
```

The call to my_method happens after the call to using, so you get the refined version of the method, just like you expect. However, the call to another_method